

쓰레기 이미지 분류 프로젝트

8단계 노트북 파이프라인 설계 요약

항목	내용
데이터셋	Kaggle mostafaabla/garbage-classification (12 클래스)
원본 규모	15,515장
정제 후	15,484장 (제외 31, 라벨 정정 46)
주 평가지표	macro-F1 (accuracy, weighted-F1, macro-AUC 병기)
모델	직접 구현 CNN (423,340 params) / ResNet18 전이학습
최종 성능	val macro-F1 0.9318 / accuracy 0.9539 / macro-AUC 0.9982 (v2)
분할	70 / 15 / 15, 그룹 인지 분할 (StratifiedGroupKFold)
노트북	8개 × 2벌 (마크다운 포함 _MD / 코드 전용)
환경	PyTorch 2.11.0+cu128, CUDA sm_120

이 문서의 범위

01~06을 v2(그룹 인지 분할)로 실행 완료한 시점의 기록이다. 표에 적힌 성능은 검증셋 기준 실측값이다. test 최종 수치와 v1(단순 분할) 세트는 아직 실행 전이므로 해당 항목은 비워 두었다.

1. 노트북 구성

각 노트북은 CSV 파일로 결과를 주고받는다. 중간 산출물이 파일로 남으므로 앞 단계를 다시 돌리지 않고 뒤 단계만 반복할 수 있다.

노트북	역할	주요 출력
01_EDA	원본 기술, 유리 3종 색상 가설 검증	index.csv, glass_hsv_samples.csv
02_Clean	중복 탐지와 라벨 정정을 manifest로 관리	manifest.csv
03_Split	두 가지 분할 전략을 각각 저장	split_v1.csv, split_v2.csv
04_Baseline	직접 만든 CNN, 불균형 대응 3종 비교	experiments.csv
05_Transfer	ResNet18 frozen / finetune 비교	experiments.csv
06_Eval	test 단 1회 적용, 확률 배열 저장	final_*.json, probs_*.npy
07_Threshold	임계값에 따른 지표 변화 3종	threshold_*.json
08_Hypothesis	가설검정 4종	각 절 p값

두 벌로 만드는 이유

_MD 버전은 제출용으로 마크다운 설명이 들어 있고, 다른 하나는 실행용으로 코드 셀만 있다. 두 버전의 코드 셀은 문자 단위로 동일하다는 것을 빌드 시 검증했다.

2. 데이터 정제 결과 (실행 완료)

처리	수량	근거
완전동일 중복 여분	19장	MD5 해시, 18그룹 37장 중 그룹당 1장만 유지
라벨 충돌 전량 제외	3장	같은 사진이 다른 폴더에 존재, 어느 쪽이 맞는지 불명
애매한 경계 사례	3장	육안으로도 판정 불가
사진이 아닌 이미지	6장	클립아트, 목업 템플릿, 스케치
라벨 정정	46장	clothes→shoes 43, shoes→clothes 1, 기타 2
근접중복 group_id	25그룹 52장	dHash 16x16 + Hamming 5 + 픽셀 검증 0.05

15,515 - 31 = **15,484장**이 최종 학습 대상이다. 팀원이 독립적으로 작성한 코드에서도 같은 값(15,493 - 라벨충돌 잔여 처리 후 동일)이 나와 탐지 로직이 교차 검증되었다.

파일을 옮기거나 지우지 않는다

모든 판정을 manifest.csv 한 장에 기록한다. filepath, orig_label, final_label, group_id, status, reason 여섯 개 열로 관리하므로 되돌릴 수 있고, 어떤 파일을 왜 뺐는지 근거가 남는다. status 플래그만 바꾸면 정제 전후를 같은 코드로 비교할 수 있다.

자주 하는 실수 하나

pandas의 duplicated(keep="first")로 중복을 지우면 짝 중 한 장만 빠진다. 클래스가 서로 다른 중복 쌍에서는 살아남은 한 장이 잘못된 라벨을 달고 학습에 들어간다. 02_Clean은 클래스 충돌 그룹을 따로 분기해 양쪽 모두 제외한다.

3. 분할 전략과 데이터 누수

MD5는 바이트가 1비트라도 다르면 중복을 못 잡는다. 같은 사진을 리사이즈하거나 다시 저장하면 해시가 완전히 달라지지만 사람 눈에는 같은 사진이다. 이런 근접중복이 train과 test로 갈라지면 모델이 시험 문제를 미리 본 셈이 되어 test 점수가 부풀려진다.

분할	방식	갈라진 그룹	비고
split_v1	train_test_split(stratify) 2단계	13그룹 26장	정제 이전의 표준 방식
split_v2	StratifiedGroupKFold 2단계	0그룹	채택

두 분할은 **같은 데이터**를 쓴다. 정제 여부가 아니라 분할 방식만 다르다. 그래서 06에서 성능 차이가 나면 원인이 하나로 특정된다. v2가 v1보다 낮게 나오는 것이 정상이며, 그것이 이 실험의 성공이다. 누수로 부풀려진 분이 빠지고 남은 것이 진짜 일반화 성능이다.

검정용 교집합

두 test는 구성이 다르므로 대응표본 검정을 바로 할 수 없다. 양쪽 test에 모두 들어간 **378장**을 따로 저장해 08의 McNemar 검정에 쓴다.

4. 평가지표 선정 근거

clothes가 전체의 약 34%다. 모든 입력을 clothes라고만 답해도 정확도 34%가 나온다. 더 중요한 것은 최소 클래스 brown-glass(약 4%)를 통째로 틀려도 전체 정확도는 4%p밖에 안 떨어진다는 점이다. **정확도는 소수 클래스의 실패를 숨긴다.**

지표	계산	성질
accuracy	맞힌 수 / 전체	다수 클래스가 지배
weighted-F1	클래스별 F1을 장수 비례 평균	다수 클래스가 지배
macro-F1	클래스별 F1을 동등 평균	소수 클래스도 같은 무게 (채택)
macro-AUC	One-vs-Rest AUC의 평균	임계값과 무관한 순위 품질

근거는 강의 자료 안에 있다

03_평가 챕터의 숫자 7 판별 반례가 같은 구조다. 전부 0으로 찍어 정확도 0.9를 얻지만 재현율은 0.0이었다. pro3(Cityscapes 분할) 실습에서는 픽셀 정확도 0.7405인데 mIoU가 0.1549로, 다수 클래스에 지배되는 지표와 클래스 균등 지표의 차이가 5배 가까이 벌어졌다. macro-F1은 이 mIoU와 정확히 같은 발상이다.

5. 모델과 실험 설계

노트북	모델	실험	설정
04	GarbageCNN	plain	불균형 대응 없음 (대조군)
04	GarbageCNN	weighted	CrossEntropyLoss(weight=w)
04	GarbageCNN	sampler	WeightedRandomSampler
05	ResNet18	frozen	backbone 동결, head만 학습
05	ResNet18	finetune	전체 학습, backbone lr/10

공통 규칙

- 체크포인트 선정은 val macro-F1 기준. 정확도가 아니다.
- 증강은 train에만 적용. val/test에 걸면 평가가 매번 달라진다.
- ColorJitter의 hue=0.0. 유리 3종은 색이 곧 정답 신호라 흔들면 정답을 지운다.
- 정규화 통계는 train split에서만 계산. 전이학습은 ImageNet 통계 사용.
- GPU를 assert로 강제. 조용한 CPU 폴백을 막는다.
- test는 06에서 단 한 번만 사용. 모델 선정은 전부 val로 한다.

전이학습에서 추가한 처리

requires_grad = False만으로는 backbone이 완전히 동결되지 않는다. BatchNorm의 이동평균은 역전파가 아니라 순전파에서 갱신되는 버퍼라 계속 변한다. frozen 실험이 진짜 특징 추출기 조건이 되도록 train()을 오버라이드해 backbone을 eval() 모드로 둔다.

6. 실험 결과 (v2, 검증셋 기준)

노트북	모델 / 실험	val macro-F1	val accuracy	best epoch
04	GarbageCNN / plain	0.7013	0.7520	14 / 15
04	GarbageCNN / sampler	0.6811	0.7051	13 / 15
04	GarbageCNN / weighted	0.6756	0.7064	15 / 15
05	ResNet18 / frozen	0.8728	0.9130	10 / 10
05	ResNet18 / finetune	0.9318	0.9539	선정

전이학습의 효과

밀바닥 학습 0.7013에서 미세조정 0.9318로 **+0.23**이다. 더 인상적인 것은 1에폭 시점의 비교다. 밀바닥 모델이 1에폭에 val macro-F1 0.4110이었는데, frozen은 **1에폭 만에 0.7669**로 밀바닥 모델의 15에폭 최종 성능을 넘겼다. frozen은 학습 대상이 6,156개로 GarbageCNN의 69분의 1인데도 그렇다. ImageNet 128만 장에서 배운 시각적 특징이 쓰레기 사진에 그대로 전이된다는 직접적인 증거다.

불균형 대응 3종 — 예상과 다른 결과

대응을 하지 않은 plain이 macro-F1에서 이겼다. 다만 클래스별로 보면 그림이 달라진다.

실험	macro-F1	최저 recall	recall 표준편차
plain	0.7013	0.4419 (plastic)	0.1422
sampler	0.6811	0.4806 (plastic)	0.1370
weighted	0.6756	0.4341 (plastic)	0.1529

sampler는 최저 클래스 재현율을 0.4419에서 0.4806으로 올리고 클래스 간 편차도 줄였다. **의도한 방향으로 정확히 작동했다**. 그러나 정밀도 손실로 macro-F1은 0.02 낮아졌다. 소수 클래스를 덜 놓치는 것과 전체 균형 성능은 별개의 목표이며, 우리는 후자를 주 지표로 삼았으므로 plain을 선택했다.

결론을 일반화하지 않는다

세 결과 모두 **과소적합 상태**에서 나왔다. train loss 1.03, val macro-F1 0.70이면 모델이 다수 클래스조차 다 못 배운 단계다. 이때 소수 클래스로 관심을 돌리라고 강제하면 양쪽 다 어중간해진다. 따라서 결론은 불균형 대응이 소용없다는 것이 아니라, 이 조건(423,340 파라미터, 15에폭, 불균형 8.77배)에서는 이득이 없었다는 것이다. weighted가 15에폭을 best epoch로 기록한 것도 수렴 전에 끊겼음을 뜻한다.

남은 오분류의 성격

finetune의 검증셋 상위 혼동은 white-glass→metal 11건, plastic→metal 9건, white-glass→plastic 7건이다. metal의 정밀도만 0.7754로 유독 낮고(다른 클래스는 0.93 이상) 재현율은 0.9304로 높다. **모델이 애매하면 metal이라고 답하는 경향**이 있다는 뜻이다. 금속성 반사광이 원인으로 보이며, 01에서 white-glass의 평균 채도와 유효 색상 픽셀 비율이 가장 낮게 측정된 것과 일치한다. 색 정보가 거의 없는 클래스라 질감과 반사에만 의존하는데 그것이 metal과 겹친다. 이 문제는 07의 클래스별 가중 argmax가 직접 겨냥하는 지점이다.

추적된 클래스 — plastic

04에서 세 실험 모두 최저 recall이 plastic이었다(0.43~0.48). 05의 finetune에서 0.8062로 거의 두 배가 됐다. 07의 임계값 분석은 F1 최저 클래스를 자동 선택하므로 이 클래스가 계속 추적된다. 04부터 07까지 한 클래스를 따라가는 서술이 가능하다.

7. 채점 기준 대응

기준	담당	내용
평가지표를 제대로 뽑고 왜 썼는지 납득	01, 06	불균형 측정 → macro-F1 채택 → accuracy / weighted-F1 / macro-AUC 병기
가설검정	08	순열검정, Kruskal-Wallis + Mann-Whitney(Bonferroni), 부트스트랩 CI, McNemar 2회
임계값에 따른 지표 변화	07	OvR 임계값 스위치, 클래스별 가중 argmax, 전역 확신도 보류

다중분류에서 임계값이란

12클래스는 argmax를 쓰므로 이진 분류의 0.5 임계값이 그대로는 존재하지 않는다. 특히 **모든 클래스에 똑같이 적용하는 조정은 예측을 하나도 바꾸지 않는다**. 온도 스케일링이 대표적이며, argmax가 단조 변환에 불변이기 때문이다. 예측이 바뀌려면 클래스마다 다르게 조정해야 한다. 07은 이 점을 먼저 코드로 확인한 뒤 세 가지 해석을 각각 다룬다.

교재 방식에서 한 가지를 고쳤다

강의 3_2와 pro2(홍부 X선)는 test 확률에서 임계값을 고르고 그 test 점수를 보고했다. 이는 test 정보가 선택에 새어드는 구조로, 같은 강의 4_8이 스택킹에서 지적한 문제와 같다. 07은 val에서 임계값을 고르고 test에는 한 번만 적용한다.

8. 강의 범위와의 대응

기법	판정	출처
CNN 직접 구현, Kaiming 초기화, 스케줄러	강의 내	05_합성곱신경망
혼동행렬, 정밀도 / 재현율 / F1	강의 내	03_평가
임계값 조정에 따른 지표 변화	강의 내	03_평가, pro2 셀 17
macro 평균 지표	강의 내	pro2 macro avg, pro3 mIoU
stratified 분할, fit은 train에만	강의 내	02_Scikit-Learn
데이터 누수, 데이터 품질의 중요성	강의 내	04_분류 7절 / 8절
전이학습 (backbone 교체 후 재학습)	강의 내	pro1 TransferModel
클래스 가중치 (pos_weight)	강의 내	pro1
ROC-AUC, PR-AUC	강의 내	pro1, pro2
MD5 / dHash / Union-Find	확장	데이터 엔지니어링
StratifiedGroupKFold의 group 개념	확장	StratifiedKFold의 연장
WeightedRandomSampler	확장	pos_weight의 다른 지점 버전
scipy 가설검정	확장	채점 기준이 요구

확장에 해당하는 항목도 대부분 배운 것의 연장이다. 특히 02_Clean의 탐지 기법은 강의 밖이지만 **판단 근거는 강의 안에 있다**. 04_분류 7절에서 신용카드 사기 데이터의 이상치 4건을 제거하자 재현율이 0.77에서 0.82로 올랐고, 그 결론은 소수 클래스에서는 데이터 한 건의 품질이 하이퍼파라미터 수십 개보다 중요하다는 것이었다. 이 프로젝트의 정제는 그 교훈의 이미지 버전이다.

9. 실행 순서와 예상 시간

단계	작업	시간	상태
1	01_EDA 실행	5분	완료
2	02_Clean 실행 (dHash 15,515장)	5분	완료 — 15,484장
3	03_Split 실행 (정규화 통계 2벌)	8분	완료 — 누수 13그룹 / 교집합 378
4	04_Baseline (v2, 3실험)	150분	완료 — 최고 plain 0.7013
5	05_Transfer (v2, 2실험)	180분	완료 — finetune 0.9318
6	06_Eval (v2)	5분	완료
7	07_Threshold, 08_Hypothesis	10분	대기
8	04~06을 v1으로 반복	180분	대기

실측 예폭 시간은 04가 200초(128px), 05의 frozen이 241초, finetune이 그 2배 남짓이었다. AMP(혼합정밀도)를 뺀 대가다. 07과 08은 06이 저장한 확률 배열을 읽으므로 모델을 다시 돌리지 않는다.

10. 조원 전달 파일

04와 05를 다시 돌리지 않고 06부터 실행할 수 있는 최소 구성이다.

위치	파일	비고
루트	01~08 노트북 8개, requirements.txt, 요약 PDF	학습 코드는 확인용
models/	transfer_v2_finetune.pt	45 MB. 나머지 .pt는 불필요
metrics/	manifest.csv, index.csv	02, 01의 결과
metrics/	split_v1.csv, split_v2.csv	03의 결과
metrics/	label_map.json, norm_stats.json	클래스 순서와 정규화
metrics/	class_weights_v1.csv, class_weights_v2.csv	04가 사용
metrics/	experiments.csv	06의 모델 선정에 필수
metrics/	glass_hsv_samples.csv	08 1절 순열검정
metrics/	mcnemar_pairs.csv	08 3절 McNemar
metrics/	clean_summary.json, split_summary.json, eda_summary.csv, glass_hue_overlap.csv	요약값
figures/	전체 폴더	발표 자료용

조원이 따로 준비할 것

Kaggle 데이터셋을 data/garbage_classification/ 아래에 12개 클래스 폴더가 오도록 푼다. split_v2.csv의 path 열이 이 상대경로를 담고 있어 구조가 다르면 파일을 찾지 못한다. torch는 requirements.txt로 설치되지 않으므로 별도 명령이 필요하다: `uv pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128`

GPU가 없는 조원이라면

06 첫 셀의 `assert torch.cuda.is_available()`에서 멈춘다. 그 경우 .pt와 데이터셋 대신 `probs_v2_val.npy`, `probs_v2_test.npy`, `preds_v2_val.csv`, `preds_v2_test.csv`, `final_v2.json`, `per_class_v2.csv`를 주면 07과 08만 3 MB로 실행할 수 있다. 이 두 노트북은 저장된 확률 배열만 읽고 모델을 부르지 않는다.

실행 중 겪은 함정

- **NUM_WORKERS는 0이어야 한다.** Windows + Jupyter에서 DataLoader 워커가 spawn으로 뜨면서 `__main__` 재임포트가 엉켜, 배치 하나 꺼내는 셀이 58분 걸렸다. 0으로 두면 5초에 끝난다.
- **experiments.csv는 구버전과 열이 다르다.** 이전 파이프라인 파일이 남아 있으면 `KeyError`가 난다. 열 구성을 확인하고 다르면 백업 후 새로 만든다.
- **06의 모델 로드 셀을 건너뛰면 NameError가 난다.** 데이터로더 셀과 추론 셀 사이에 있다.

11. 남은 한계

보고서와 발표에 명시할 항목이다. 감추는 것보다 아는 것을 드러내는 편이 낫다.

- **탐지 threshold는 임의값이다.** Hamming 5, 픽셀차 0.05는 시행착오로 정한 값이지 최적화된 값이 아니다. 느슨하면 오탐이 늘고 조이면 진짜 중복을 놓친다.
- **라벨 검수 범위가 클래스마다 다르다.** 확신도 기반 진단은 모델이 확신하며 틀린 것만 후보로 올리므로, 모델이 애초에 잘 못 배우는 유형의 오류는 놓친다. clothes에 정정이 몰린 것도 실제로 오류가 많아서인지 검수가 그쪽에 집중돼서인지 구분되지 않는다.
- **문서 간 수치 불일치가 있다.** label_review_report.md는 검토 건수를 232건으로 적고 있으나 1.ipynb 코드는 CONF_THRESHOLD = 0.9 고정이라 204건이 재현된다. 0.7까지 확장한 실행 기록을 찾거나 문서를 정정해야 한다.
- **단일 시드 단일 학습이다.** 08의 McNemar는 시드 하나의 결과를 비교한 것이다. 엄밀하게는 여러 시드로 반복해야 한다.
- **에폭 상한이 제약이었다.** 04의 weighted는 15에폭이 best epoch였고 05의 frozen은 10에폭이 best epoch였다. 둘 다 아직 오르는 중에 끝났다. 더 돌렸다면 순위가 바뀌었을 수 있다.
- **누수 효과 크기가 작다.** 26장은 test의 1.1%라 공통 378장에서는 검정력이 부족하다. 기각 실패가 나와도 그것은 차이가 없다는 뜻이 아니라 이 표본으로 확인할 수 없다는 뜻이다.
- **임계값 곡선이 평평할 수 있다.** 모델이 과확신 상태면 확률이 0과 1에 몰려 조정 여지가 작다. 그 자체가 측정된 결과이며, pro2의 페럼 모델과 나란히 놓으면 같은 기법이 모델 상태에 따라 다르게 작동한다는 결론이 된다.

본 문서는 v2 세트 06까지 실행 완료 시점 기준이다. 표의 성능은 검증셋 실측값이며, test 최종 수치와 v1 비교 결과는 실행 후 갱신한다.